



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 13

по курсу «Языки и методы программирования»

«Сборка MediaPipe из исходного кода с использованием Docker и Bazel»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Что такое MediaPipe	1
Docker-образ для сборки	1
Сборка примеров	3
Сборка hello_world	3
Сборка hand_tracking_cpu	3
Анализ результатов	4
Вывод	4

Цель работы

Ознакомиться с процессом сборки фреймворка MediaPipe из исходного кода с использованием Docker-контейнеризации и системы сборки Bazel. Выполнить сборку примеров desktop-приложений (hello_world и hand_tracking) внутри Docker-образа.

Что такое MediaPipe

MediaPipe — это открытый фреймворк от Google для построения конвейеров машинного обучения (ML) на различных платформах: мобильные устройства (Android, iOS), веб, десктоп, IoT. MediaPipe предоставляет готовые решения для:

- распознавания рук (Hand Tracking);
- детекции лиц (Face Detection);
- определения позы (Pose Estimation);
- сегментации изображений;
- отслеживания объектов и других задач компьютерного зрения.

Фреймворк использует Bazel в качестве основной системы сборки. Исходный код написан на C++ с поддержкой Java (Android), Objective-C (iOS) и Python.

Docker-образ для сборки

Для изолированной сборки MediaPipe используется официальный Dockerfile на базе Ubuntu 22.04. В образ устанавливаются все необходимые зависимости:

- компиляторы GCC/G++ и Clang 16;
- OpenCV (библиотека компьютерного зрения);
- Java 21 (требуется для Bazel);
- Python 3 и пакеты (absl-py, numpy, opencv-contrib-python, protobuf, tensorflow и др.);
- Bazel 7.4.1 (система сборки);
- утилиты ffmpeg, mesa (для работы с графикой).

Dockerfile

```
FROM ubuntu:22.04

WORKDIR /mediapipe

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential gcc g++ ca-certificates curl ffmpeg git wget \
    unzip \
    nodejs npm python3-dev python3-opencv python3-pip \
    libopencv-core-dev libopencv-highgui-dev libopencv-imgproc-dev \
    \
    libopencv-video-dev libopencv-calib3d-dev libopencv-features2d- \
    dev \
    software-properties-common && \
    apt-get update && apt-get install -y openjdk-21-jdk && \
    apt-get install -y mesa-common-dev libegl1-mesa-dev \
    libgles2-mesa-dev mesa-utils && \
    apt-get clean && rm -rf /var/lib/apt/lists/*

# Установка Clang 16
RUN wget https://apt.llvm.org/llvm.sh
RUN chmod +x llvm.sh
RUN ./llvm.sh 16
RUN ln -sf /usr/bin/clang-16 /usr/bin/clang
RUN ln -sf /usr/bin/clang++-16 /usr/bin/clang++
RUN ln -sf /usr/bin/clang-format-16 /usr/bin/clang-format

# Установка Python-зависимостей
RUN pip3 install --upgrade setuptools wheel future \
    absl-py "numpy<2" jax[cpu] opencv-contrib-python \
    protobuf==3.20.1 \
    six==1.14.0 tensorflow tf_slim

# Установка Bazel 7.4.1
ARG BAZEL_VERSION=7.4.1
RUN mkdir /bazel && \
    wget --no-check-certificate -O /bazel/installer.sh \
    "https://github.com/bazelbuild/bazel/releases/download/\
    ${BAZEL_VERSION}/bazel-${BAZEL_VERSION}-installer-linux-x86_64.sh" \
    && \
    chmod +x /bazel/installer.sh && /bazel/installer.sh && \
    rm -f /bazel/installer.sh

COPY . /mediapipe/
```

Сборка образа выполняется командой:

Сборка Docker-образа

```
docker build -t mediapipe:latest .
```

Образ получается объёмным — около 15.6 ГБ на диске (4.02 ГБ в сжатом виде), что обусловлено большим количеством зависимостей (TensorFlow, Bazel, OpenCV, библиотеки для GPU).

Сборка примеров

После создания образа был запущен интерактивный сеанс внутри контейнера:

Запуск контейнера

```
docker run -it --name mediapipe mediapipe:latest /bin/bash
```

Сборка hello_world

Первым был собран минимальный пример hello_world для проверки работоспособности Bazel и инструментария:

Сборка hello_world

```
bazel build --define MEDIAPIPE_DISABLE_GPU=1 \
mediapipe/examples/desktop/hello_world
```

Сборка прошла успешно. Было выполнено 3259 действий (actions), из которых все завершились без ошибок. Бинарный файл был размещён в директории bazel-bin/mediapipe/examples/desktop/hello_world/.

Сборка hand_tracking_cpu

Следующим этапом была выполнена сборка примера Hand Tracking (распознавание рук) с использованием CPU:

Сборка hand_tracking_cpu

```
CC=/usr/bin/clang CXX=/usr/bin/clang++ \
bazel build -c opt --define MEDIAPIPE_DISABLE_GPU=1 \
--copt=-I/usr/include/opencv4 \
--cxxopt=-I/usr/include/opencv4 \
mediapipe/examples/desktop/hand_tracking:hand_tracking_cpu
```

Флаги сборки:

- CC и CXX — явное указание компилятора Clang 16;

- `-c opt` — оптимизированная сборка;
- `--define MEDIAPIPE_DISABLE_GPU=1` — отключение поддержки GPU;
- `--copt=-I/usr/include/opencv4` — указание путей к заголовочным файлам OpenCV.

Из 3259 действий успешно выполнилось 3018 (92.6%). Сборка завершилась с ошибкой на этапе `EncodeProto`, где утилита `protoc` не смогла найти разделяемую библиотеку `libstdc++.so.6`.

Фрагмент лога ошибки

```
ERROR: .../hand_landmark:hand_landmark_model_loader_graph failed:
bazel-out/.../bin/external/protobuf~/.protoc:
error while loading shared libraries:
libstdc++.so.6: cannot open shared object file:
No such file or directory
```

Причина: в образе Ubuntu 22.04 установлена библиотека `libstdc++6`, но Bazel при сборке `protoc` из исходников использует кастомный toolchain Clang 16, который ищет библиотеку по нестандартному пути. Проблема решается установкой пакета `libstdc++-12-dev` или добавлением пути к библиотеке в `LD_LIBRARY_PATH`.

Анализ результатов

Пример	Результат	Выполнено действий	Статус
hello_world	Бинарный файл собран	3259 / 3259 (100%)	Успешно
hand_tracking_cpu	Ошибка protoc + libstdc++	3018 / 3259 (92.6%)	Не завершён

Несмотря на частичную неудачу, 3018 действий были выполнены корректно, что свидетельствует о работоспособности инструментария и правильности Docker-окружения. Ошибка носит локальный характер, связана с конфигурацией toolchain в контейнере и не является критической для понимания процесса сборки MediaPipe.

Вывод

В ходе лабораторной работы был изучен процесс сборки фреймворка MediaPipe из исходного кода с использованием Docker и Bazel:

- создан Docker-образ на основе Ubuntu 22.04 с установкой Clang 16, OpenCV, Java 21, Python-зависимостей и Bazel 7.4.1;
- внутри контейнера выполнена сборка примера `hello_world` (успешно);
- выполнена сборка `hand_tracking_cpu` (частично, с ошибкой `libstdc++`).

Получен практический опыт работы с системой сборки Bazel, настройки Docker-окружения для C++ проектов машинного обучения и диагностики ошибок при сборке крупных фреймворков.